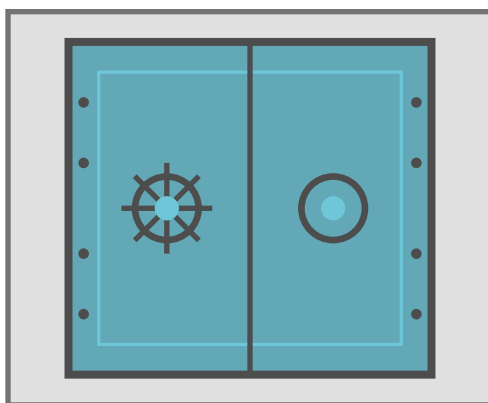


Criptografía y Seguridad

Seguridad en aplicaciones

Autenticación



Apunte integral — Teoría + Práctica

Clase 7 · Capítulos 12–13, *Computer Security: Art and Science* (M. Bishop)

Instituto Tecnológico de Buenos Aires (ITBA)

Documento elaborado a partir de las transcripciones de la clase teórica (Prof. Leandro Suárez) y de la clase práctica, junto con las diapositivas oficiales de ambas.

Índice

1. Introducción y contexto	2
2. Conceptos fundamentales	2
2.1. Autenticación $\neq$ Autorización	2
2.2. El proceso de autenticación	3
3. Sistema de autenticación	3
3.1. Ejemplo: autenticación con contraseñas	4
4. Información de autenticación: los factores	4
4.1. Algo que conozco (conocimiento compartido)	4
4.2. Algo que tengo (posesión física)	5
4.3. Algo que soy (biométrico / inherente)	5
4.4. Contexto	5
5. Claves (contraseñas)	5
5.1. Tipos de claves	6
5.2. Almacenamiento de claves	6
5.2.1. Texto plano — NO RECOMENDADO	6
5.2.2. Archivo cifrado	6
5.2.3. Ejemplo: sistema Unix (legacy)	6
6. Ataques a un sistema de autenticación	7
6.1. Adivinando claves: offline vs. online	7
6.2. Cómo se “mejoran” los ataques	7
7. Prevenciones	8
7.1. Prevenciones generales	8
7.2. Subir la complejidad: fórmula de Anderson	8
7.3. Selección de claves — alternativas	9
7.4. Revisión proactiva de claves	9
7.5. Expiración de claves	9
8. Almacenamiento seguro: Salting y PBKDF2	9
8.1. Salting	9
8.2. PBKDF2 (PKCS #5 v2.0)	10
9. Protocolos sin transmitir la clave	10
9.1. Challenge–Response (pregunta–respuesta)	11
9.2. EKE — Encrypted Key Exchange	11
10.OTP — Claves de un solo uso	11
10.1. HOTP — RFC 4226 (basado en HMAC)	12
10.2. TOTP — RFC 6238 (basado en tiempo)	12
11.Autenticación multifactor (MFA)	12
12.Autenticación remota y federada (SSO)	13
Lectura recomendada	13

1 Introducción y contexto

Esta clase abre el bloque de **Seguridad en aplicaciones** y se concentra en la **autenticación**. Conviene situarla respecto de lo visto previamente.

Triada CIA — el marco de toda la seguridad

Toda política de seguridad busca *forzar* el cumplimiento de tres propiedades sobre la información de un sistema:

- **Confidencialidad** (*Confidentiality*): protege las *lecturas*; que quien lee la información sea quien debe ser.
- **Integridad** (*Integrity*): protege las *modificaciones/escrituras*; que nadie sin permiso pueda alterar la información.
- **Disponibilidad** (*Availability*): que quien debe acceder pueda hacerlo *en el momento* en que lo necesita.

De las políticas de seguridad surgen los **modelos de seguridad** y los esquemas de **control de acceso** (mandatorios —MAC— y discrecionales —DAC—), vistos en la clase anterior. En el fondo, el control de acceso es un conjunto de reglas que definen *si un sujeto puede acceder a un objeto y de qué forma*.

Pero esas reglas presuponen algo: que en el sistema hay **sujetos** (entidades externas, normalmente físicas: personas, dispositivos) que interactúan con él. Para poder aplicar las reglas, primero hay que *representar internamente* a esas entidades y, sobre todo, *comprobar que cada sujeto es quien dice ser*. Ese es exactamente el problema de la autenticación.

¿Por qué importa tanto?

La autenticación es uno de los aspectos **más atacados** de un sistema. Una vez que un atacante logra impersonar a un usuario, el sistema —que rara vez tiene mecanismos fuertes para detectar que “ese ya no sos vos”— le permite hacer todo lo que el usuario legítimo podría. Es un punto crítico de la seguridad de las aplicaciones.

2 Conceptos fundamentales

Autenticación

Autenticación es el proceso de **asociar una identidad a un principal**.

- **Entidad**: el sujeto externo y físico (una persona, un dispositivo).
- **Identidad**: la representación computacional de una entidad dentro del sistema (p. ej. un usuario en el sistema *Pampero*).
- **Principal**: la entidad única ya representada internamente (usuario, sujeto). *Un mismo principal puede tener distintas identidades* (roles), porque los permisos dependen del rol que cumple en cada momento.

La identidad se usa luego para el **control de accesos** y la **asignación de recursos**.

2.1 Autenticación ≠ Autorización

Son dos momentos distintos, ambos deben estar bien resueltos:

- **Autenticación:** *¿quién sos?* Verifica la identidad.
- **Autorización:** ya autenticado, *¿podés acceder a este recurso?* Pertenece al control de acceso. *Estoy autenticado pero no necesariamente autorizado para todo.*

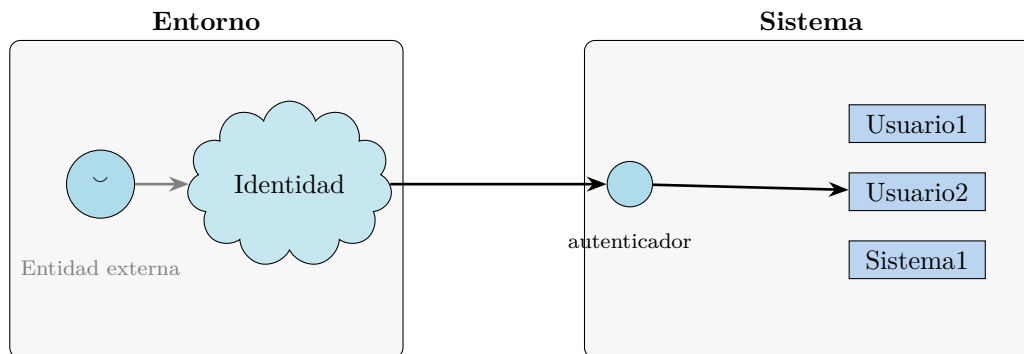


Figura 1: La entidad externa proyecta una *identidad*; el *autenticador* la asocia a un *principal* interno del sistema. (Recreación de la diapositiva 2.)

2.2 El proceso de autenticación

La entidad debe **brindar información que confirme su identidad**. Los pasos:

1. La entidad **provee** la información que acredita su identidad (p. ej. la contraseña).
2. Se **analiza** esa información comparándola con la que el sistema tiene guardada.
3. Se **determina** si la información está asociada a esa entidad $\Rightarrow$ verdadero / falso.

Como *consecuencias*, el sistema debe (a) **almacenar** información de cada entidad y (b) **contar con mecanismos** para procesar esa información y hacer la corroboración.

3 Sistema de autenticación

Una cosa es el *proceso* de autenticación y otra, más amplia, el *sistema*. El sistema de autenticación se modela —al igual que un criptosistema— como una **tupla de cinco componentes**. Si cualquiera de ellos falla, falla todo el sistema.

Componentes del sistema de autenticación

$$\langle \mathcal{A}, \mathcal{C}, \mathcal{F}, \mathcal{L}, \mathcal{S} \rangle$$

$$\mathcal{A} = \{a\}$$

Información de autenticación. La que provee la entidad externa para probar su identidad (contraseña, huella, token). *El sistema no la controla: viene dada desde afuera.*

$$\mathcal{C} = \{c\}$$

Información complementaria. La que el sistema *almacena* para identificar al principal (p. ej. el *hash* de la contraseña, o las *minucias* de una huella).

$$\mathcal{F} = \{f : \mathcal{A} \rightarrow \mathcal{C}\}$$

Funciones de complementación. Derivan c a partir de a . Idealmente *no invertibles* (de c no se debe poder volver a a).

$$\mathcal{L} = \{\ell : \mathcal{A} \times \mathcal{C} \rightarrow \{0, 1\}\}$$

Funciones de autenticación. Dado un a y el c guardado, deciden si la asociación es válida. Es la parte con la que el usuario *interactúa* (el *login*).

$$\mathcal{S} = \{s\}$$

Funciones de selección. Crean o alteran $\mathcal{A}$ y $\mathcal{C}$ (alta de usuario, cambio de contraseña). Son las más *administrativas*.

Distinción F vs. L (consulta de la clase teórica)

- F **deriva**: a partir de la información de autenticación produce la complementaria ($A \rightarrow C$). Ej.: “calculá el hash de la contraseña”.
- L **corrobor**a: dada una *nueva* información de autenticación, verifica si existe una complementaria correspondiente en el sistema ($A \times C \rightarrow \{T, F\}$).

3.1 Ejemplo: autenticación con contraseñas

Modelo ilustrativo (*no* el ideal de almacenamiento, como veremos):

$$\begin{aligned} \mathcal{A} &= \{x \mid x \text{ es una clave}\} & \mathcal{L} &= \{==\} \quad (\text{válido si } F(a) = c) \\ \mathcal{C} &= h(\mathcal{A}) & \mathcal{S} &= \{\text{adduser}(), \text{removeuser}(), \text{passwd}()\} \\ \mathcal{F} &= \{h(x)\} \text{ (función de hash)} \end{aligned}$$

En la práctica se escribe equivalentemente: $H : \mathcal{A} \rightarrow \mathcal{C}$, $H(p) = \text{hash}(p)$ y $\text{eq} : \mathcal{A} \times \mathcal{C} \rightarrow \{\text{true}, \text{false}\}$ con $\text{eq}(p, h) = \text{true} \iff \text{hash}(p) = h$.

4 Información de autenticación: los factores

La información de autenticación es provista por la entidad externa y posee **distintos grados de confianza**. No es lo mismo que alguien diga una contraseña (que pudo haber escuchado) a que muestre su cara. Las **fuentes** o **factores** indican la *naturaleza* de la información. Hay tres factores clásicos —“algo que sé / tengo / soy”— más el contexto.

Factor	Se basa en...	Ejemplos
Algo que conozco	un <i>secreto compartido</i>	clave, <i>passphrase</i> , pregunta secreta
Algo que tengo	un <i>elemento físico</i> (posesión)	smart card / USB token, RSA-ID, celular, tarjeta de coordenadas (token grid)
Algo que soy	características <i>intrínsecas</i> (biométrico)	huella, retina, voz, cara
Contexto	el <i>contexto</i> de la conexión	red de origen, país/ciudad/geo, host/terminal, día y hora, <i>velocity</i>

4.1 Algo que conozco (conocimiento compartido)

Es el factor más antiguo, estudiado y atacado. Basa la identificación en un **secreto compartido** entre las dos partes (clásicamente, la contraseña).

- Requiere **almacenar de manera segura** el secreto y depende de su **confidencialidad**.
- Sirve de **base** a muchos otros mecanismos.
- Las *pass-phrases* y *preguntas secretas* están quedando en desuso (mala práctica: a veces son *más* adivinables que una contraseña).
- Si el usuario anota la clave y un tercero la ve, se **vulneró al usuario, no al sistema**. Distinto —y grave— es que se infiera la clave de la información *almacenada*.

4.2 Algo que tengo (posesión física)

Análogo a la llave de una casa: basa la autenticación en un elemento físico que se posee.

- Requiere el **cuidado** del elemento; puede **dañarse, robarse o clonarse**, y la información *en tránsito* puede ser suplantada.
- Ejemplos: *Smart cards*, *YubiKey* (USB que se conecta y habilita el acceso), el celular, y los **tokens físicos** tipo RSA con un visor que muestra un código que *rota* cada cierto tiempo.
- La rotación del código mitiga el robo: si se intercepta, tiene una ventana muy corta para operar.

4.3 Algo que soy (biométrico / inherente)

Características físicas **difíciles de modificar o perder**.

- **No son 100 % eficaces**: no hay lector perfecto, y un mismo C podría validar dos A distintos (caso del hermano/primo que desbloquea el celular).
- Las primeras huellas en celulares fallaban tanto que se *aumentaba el umbral* de error y se aceptaban lecturas imperfectas $\Rightarrow$ menos seguridad.
- **Asumen que el dispositivo de lectura no puede ser manipulado**. Por eso se agregan cámaras u otros mecanismos contra suplantación (foto ante un lector de cara, interceptación del dispositivo, etc.).

El chip biométrico del celular

Los celulares tienen un **chip dedicado** para leer y almacenar la información biométrica, *por fuera del kernel* del sistema operativo. El SO **no puede acceder** a ese chip: solo recibe un OK / FAIL. Cuando ponés la cara, el SO solo le pregunta a ese subsistema si la información coincide y toma una decisión.

4.4 Contexto

No suele usarse como factor único sino como **complemento**. Basa la identificación en el contexto de la conexión y aplica **reglas positivas o negativas**:

- **Positiva**: “un operador de un centro de cómputos debe conectarse desde la red privada”.
- **Negativa** (*velocity*): si me conecto en Buenos Aires y dos horas después desde Ámsterdam (a ~ 8 h de vuelo), *hay menos chances de que sea yo* $\Rightarrow$ se pide una habilitación o un factor adicional.

5 Claves (contraseñas)

Las claves se estudian en profundidad por ser el mecanismo **más utilizado**. A diferencia de las claves criptográficas, **no son controladas por el sistema sino por el usuario** y *no* suelen generarse aleatoriamente. Por eso el sistema toma decisiones (restricciones) para resguardarse de ataques.

5.1 Tipos de claves

- **Secuencias de caracteres:** con restricciones (largo mínimo, dígitos, mayúsculas, símbolos); generadas aleatoriamente, por el usuario o de manera asistida (el navegador ofrece generarlas y guardarlas).
- **Secuencias de palabras** (*pass-phrases*): más largas; en desuso porque suelen ser *más adivinables*.
- **Algorítmicas:** *pregunta-respuesta* (challenge-response) y *claves de un solo uso* (OTP — *one time passwords*).

OTP ≠ One Time Pad

La sigla OTP de “*one time password*” **no** es el *One Time Pad* (cifrado de Vernam). Tampoco hay que confundir las OTP con los *recovery codes*: estos últimos son códigos de un solo uso que se “van quemando” para recuperar la cuenta si se pierde el dispositivo del segundo factor.

5.2 Almacenamiento de claves

5.2.1 Texto plano — NO RECOMENDADO

Puede estar en un archivo o base de datos y ser accedido por fuera del sistema; no puede garantizarse la confidencialidad. Si se roba la base, se llevan *todas* las contraseñas. Riesgo adicional poco mencionado: los **propios administradores/desarrolladores** con acceso a la base podrían leer secretos y hacer “ataques hormiga” imperceptibles. La mitigación de fondo es organizacional: **concientización**, políticas que fuercen buenas prácticas, y herramientas que *detectan secretos en plano* en bases y repositorios.

5.2.2 Archivo cifrado

Requiere una clave para acceder a las contraseñas. Esa clave puede estar en un archivo de configuración, en el propio ejecutable, ingresarse al iniciar el sistema, o vivir en un **HSM** (*Hardware Security Module*, dispositivo criptográfico especializado).

Solo recomendado si es requisito acceder a la clave *original* — por ejemplo, para reenviarla a un sistema *legacy* (típico en bancos, PCI/tarjetas de crédito) que exige recibir la contraseña original y validarla.

5.2.3 Ejemplo: sistema Unix (legacy)

Unix marcó un precedente: en vez de cifrar, empezó a guardar una **prueba de que se conoce el secreto, sin guardar el secreto**. Se almacena por cada usuario el resultado de **una de 4096 funciones de hash**, elegida aleatoriamente:

$$\begin{aligned}\mathcal{A} &= \{\text{secuencias de hasta 8 caracteres}\} \\ \mathcal{C} &= \left\{ \underbrace{\quad \text{xx} \quad}_{\text{función (0-4095)} - 2 \text{ char resultado}} \underbrace{\text{HHHHHHHHHHH}}_{11 \text{ char}} \right\} \\ \mathcal{F} &= \{ \text{DES}_{\text{xx}}(\cdot) \} \quad (\text{variante de DES no reversible}) \\ \mathcal{L} &= \{\text{login, su, sudo, \dots}\} \quad \mathcal{S} = \{\text{passwd, adduser, \dots}\}\end{aligned}$$

El indicador **xx** señala *qué* función se usó; el resultado se obtiene con una variante de DES *no* invertible. Más adelante surgieron algoritmos que emulan este comportamiento.

6 Ataques a un sistema de autenticación

Objetivos enfrentados

- **Del sistema:** identificar *correctamente* a las entidades.
- **Del atacante:** ser identificado como una entidad que no es (**impersonarla**).

Mecanismo del atacante: encontrar $a \in \mathcal{A}$ tal que, para algún $f \in \mathcal{F}$, $f(a) = c$ y c esté asociado a una entidad.

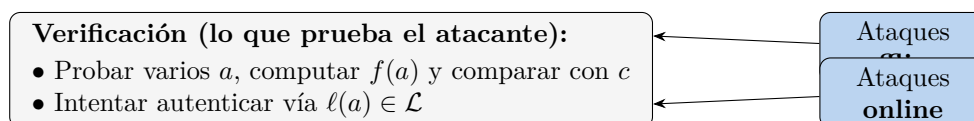


Figura 2: Atacar la función f (offline) o la función ℓ (online). (Recreación de la diapositiva 15.)

6.1 Adivinando claves: offline vs. online

- **Offline:** se conoce f y c y se prueban distintos a hasta que $f(a) = c$. Caso típico: *me robé la base de datos* (que contiene la información complementaria, p. ej. los hashes, no las contraseñas). Herramientas: **crack**, **John the Ripper** (archivo `/etc/shadow` de Linux), **ophcrack** (archivo SAM de Windows).
- **Online:** se ataca directamente la función ℓ (el *login*), probando distintos a hasta pasar la autenticación —típicamente con cuentas conocidas (**root**, **administrator**, **guest**)—. Se juega con el inspector, con un cliente tipo *Postman*, buscando vulnerabilidades del servidor.

6.2 Cómo se “mejoran” los ataques

Como las claves *no* son aleatorias, el atacante tiene una posición ventajosa (“hasta ahora se asumía búsqueda aleatoria”):

- **Diccionarios de palabras:** la gente usa palabras existentes, nombres propios.
- **Diccionarios de claves descubiertas:** históricamente hubo *leaks/breaches* de bases con contraseñas; muchas se reutilizan. Existen diccionarios especializados.
- **Transformaciones simples:** sufijos, prefijos, reemplazos ($1 \rightarrow 1$, $o \rightarrow 0$, vocales por números), palabras espejadas, dos palabras combinadas. “Somos humanos, tenemos las mismas ideas.”
- **Información de contexto** (la peor): nombre, usuario, DNI, fecha de nacimiento.

Catálogo de ataques (clase práctica)

Ingeniería social	<i>Phishing, man-in-the-middle.</i> El más frecuente y el que más “hace caer” a la gente: termina en una impersonación del usuario.
Fuerza bruta	Prueba exhaustiva de todas las claves posibles. Poco frecuente por su costo.
Diccionario	Lista de palabras comunes (más frecuente que la fuerza bruta pura).
Tablas arcoíris (Rainbow)	Diccionario <i>de hashes ya calculados</i> .
Password stuffing	Reutilizar credenciales robadas (por phishing, MIM, ingeniería social) en otros sitios donde la víctima repitió la contraseña.

7 Prevenciones

7.1 Prevenciones generales

Esconder información para que a , c o f no sean conocidas *simultáneamente*:

- Por lo general f **es conocida o puede conocerse** (hay pocos algoritmos estándar; incluso por *reverse engineering*).
- Es más fácil proteger c que a (que está en manos de la entidad externa). Ejemplo: `/etc/shadow` en Unix, accesible solo por `root`.

Prevenir el uso de la función de autenticación ℓ (ataques *online*):

- **Exponential back-off**: ante cada falla se aumenta exponencialmente el tiempo de reintento (1 s, 10 s, ...) para frenar la fuerza bruta.
- **Deshabilitar principales forzados** (*brute-forced*). Mecanismo “borde”: un malicioso puede afectar la *disponibilidad* de un usuario solo para molestar (algunos bancos aún bloquean tras 5 intentos).
- **Jailing / Honeypot**: al detectar un ataque, se *switchea* la función de autenticación por otra falsa que deja pasar al atacante a un **ambiente controlado** mientras el equipo de seguridad lo *tracea*. (“El bote de miel para el oso.”) Costoso; lo usan agencias gubernamentales y grandes corporaciones.
- **Captchas**: baratos (se ofrecen como servicio), pero cada vez más vulnerados por bots e IA.

7.2 Subir la complejidad: fórmula de Anderson

Fórmula de Anderson

$$P = \frac{T \cdot G}{N}$$

P probabilidad de adivinar una clave

T tiempo dedicado a las pruebas

G número de pruebas por segundo

N tamaño del espacio de claves

Permite **estimar el tiempo** de un ataque (o derivar requisitos de longitud).

Ejercicio 1. Claves numéricas de 10 dígitos ($N = 10^{10}$), $G = 10\,000$ claves/s, $P \geq 0.5$:

$$P \geq \frac{TG}{N} \Rightarrow T \leq \frac{PN}{G} = \frac{0.5 \cdot 10^{10}}{10^4} = 5 \times 10^5 \text{ s} \approx \mathbf{6 \text{ días}}.$$

Ejercicio 2. Claves de 8 letras (26 caracteres, $N = 26^8$), $G = 10\,000$ /s, $P \geq 0.5$:

$$T \leq \frac{0.5 \cdot 26^8}{10^4} \approx 1\,044\,135 \text{ s} \approx \mathbf{121 \text{ días}}.$$

Con menos longitud pero mayor alfabeto, la complejidad sube muchísimo.

Ejercicio 3. ¿Qué **longitud mínima** L se requiere con claves alfanuméricas (36 caracteres) y $G = 100\,000$ /s para que $P < 0.5$ de hallarla en *menos de un año*?

$$P \geq \frac{TG}{N} \Rightarrow N \geq \frac{TG}{P} = \frac{(365 \cdot 24 \cdot 60 \cdot 60) \cdot 10^5}{0.5} \Rightarrow N = 36^L \geq 6.3 \times 10^{12}$$

$$L \geq \log_{36}(6.3 \times 10^{12}) \approx 8.22 \Rightarrow \boxed{L \geq 9}.$$

7.3 Selección de claves — alternativas

- **Selección aleatoria:** condición ideal (toda clave equiprobable), pero difícil de memorizar (fL3K&8%j).
- **Claves pronunciables:** palabras sin sentido formadas por fonemas (*helgoret*, *mipoterjo*, *jusacila*); fáciles de memorizar pero **reducen mucho el espacio** de claves.
- **Permitir que el usuario elija:** tiende a claves fáciles de adivinar. Los *password managers* ayudan, pero concentran el riesgo en la **clave maestra** (si la roban, caen todas).

7.4 Revisión proactiva de claves

Analizar la clave **al momento de crearla** para detectar y rechazar claves “fáciles”. Debería: aplicar reglas sobre palabras, buscar en diccionarios y en *leaks/breaches*, detectar patrones y usar información del usuario y del contexto (rechazar nombre/fecha de nacimiento).

7.5 Expiración de claves

Forzar el cambio tras cierto tiempo o evento **limita el daño** de una clave comprometida. Requisitos:

- Evitar el **reúso** (recordar y bloquear las N últimas).
- Evitar cambiarla varias veces seguidas para “saltar” el bloqueo.
- **Dar tiempo** para pensar una nueva: *no* forzar el cambio en el login y **avisar con anticipación**.

El riesgo de la rotación mal hecha

Un usuario apurado al que se le venció la clave elige una *sencilla* sin pensarla $\Rightarrow$ **perjudica** la seguridad en lugar de ayudarla. De ahí la importancia del preaviso (“en 7 días vence tu contraseña...”).

8 Almacenamiento seguro: Salting y PBKDF2

8.1 Salting

Práctica **obligatoria** hoy: agregar una *perturbación* (*salt*) a la información que se guarda de forma complementaria.

Salting

Escenario: un atacante consigue $c_1, c_2, \dots, c_n$. Sin salt, cada prueba $a_i \mapsto f(a_i)$ puede compararse *en paralelo* contra todos los c_j .

Objetivo: aumentar el tiempo requerido para adivinar claves.

Método: introducir una perturbación

$$f(a) = x \parallel f'(a, x)$$

donde cada c se calcula con una perturbación x *distinta*. Así el atacante **no puede reutilizar** $f(a)$ para buscar varias cuentas en paralelo.

Beneficios concretos:

- **No hay patrones:** dos contraseñas iguales *no* producen el mismo c .
- Hay que obtener el salt **de cada** usuario; el ataque **no se paraleliza**.
- El salt suele elegirse aleatoriamente y **embeberse** en el hash.
- Estos algoritmos están *diseñados para ser costosos* (p. ej. ≥ 100 ms por evaluación): al usuario solo le importa una vez, al atacante le complica todo.

8.2 PBKDF2 (PKCS #5 v2.0)

Password-Based Key Derivation Function 2: **no es un hash**, es una **función de derivación**. Muy usada y robusta (junto con *bcrypt*, *scrypt*).

PBKDF2 — Idea y algoritmo

Sea *pass* una contraseña, S un *salt* y PRF una función pseudoaleatoria (criptosistema simétrico o MAC). Se aplica **iterativamente**:

$$u_1 = \text{PRF}(\text{pass}, S), \quad u_2 = \text{PRF}(\text{pass}, u_1), \quad \dots, \quad u_j = \text{PRF}(\text{pass}, u_{j-1})$$

$$\text{Resultado: } u_1 \oplus u_2 \oplus \dots \oplus u_j$$

Forma general: $K = \text{PBKDF2}(\text{prf}, \text{pass}, \text{salt}, c, \text{len})$, con $K = T_1 \parallel T_2 \parallel \dots \parallel T_n$ ($n = \text{len}/|\text{prf}|$) y

$$T_i = u_1 \oplus u_2 \oplus \dots \oplus u_c, \quad u_1 = \text{PRF}(\text{Pass}, \text{Salt} \parallel \text{INT32_BE}(i)), \quad u_k = \text{PRF}(\text{Pass}, u_{k-1}).$$

El número de **rondas** c (la J) es un parámetro: a más rondas, más costo para el atacante (al usuario solo le cuesta una vez). El *XOR* final ayuda a **uniformizar** los bits y evitar patrones detectables.

TIP — en Java, JCE implementa PBKDF2:

```
PBEKeySpec spec = new PBEKeySpec(password, salt, iterations, bytes * 8);
SecretKeyFactory skf = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1");
byte[] key = skf.generateSecret(spec).getEncoded();
```

No hay que implementar nada: la librería estándar de seguridad ya lo provee.

9 Protocolos sin transmitir la clave

Problema: autenticar a un usuario remoto suele requerir *enviar* la clave; aun sobre un canal seguro, descubrir una comunicación antigua puede comprometerla. **Solución:** *no enviar la clave*.

9.1 Challenge–Response (pregunta–respuesta)

Se valida que el usuario *conoce* el secreto **sin transmitirlo**.

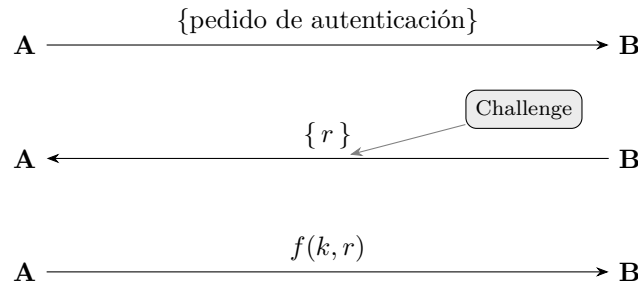


Figura 3: *B* envía un *challenge* aleatorio r ; *A* responde $f(k, r)$ aplicando una función preacordada sobre su clave k y r , sin enviar k . (Recreación de la diapositiva 29.)

9.2 EKE — Encrypted Key Exchange

Objetivo: impedir que un atacante adivine claves capturadas en la red. **Mecanismo:** realizar el diálogo *challenge–response* sobre un **canal encriptado** con clave k_s . Así el atacante **no tiene forma de saber si una clave es correcta**.

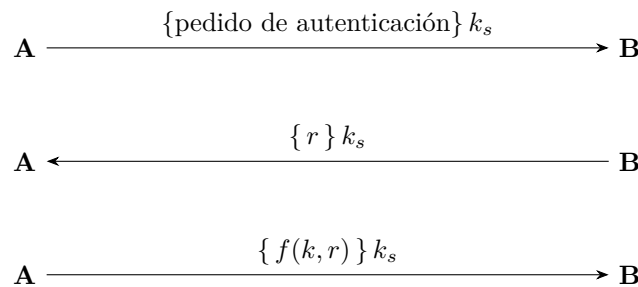


Figura 4: Todo el intercambio viaja cifrado bajo k_s . (Recreación de la diapositiva 30.)

Passkeys (mención)

Las *passkeys* —cada vez más relevantes— son un caso de challenge–response: el sistema te envía un *token* único que firmás con un secreto que solo vos comprobás (PIN, cara, huella). **No enviás ningún dato secreto:** solo devolvés el challenge firmado, y el servidor lo verifica. Por debajo hay un esquema de **clave pública/privada**.

10 OTP — Claves de un solo uso

Concepto de OTP

Generar claves que sirven **una única vez** (válidas para una sola sesión/transacción). *A* y *B* comparten una clave K y un contador C :

$$(K, C) \rightarrow \text{OTP} \rightarrow (\text{otp}, C')$$

Cuestiones a resolver: la *generación pseudoaleatoria* y la *sincronización* entre usuario y sistema. Dos variantes estandarizadas: **HOTP** (*hmac based*) y **TOTP** (*time based*). La clave secreta es criptográfica, generada aleatoriamente; al enrolar se escanea el QR (o se ingresa el

código), que es la *representación de la clave*.

10.1 HOTP — RFC 4226 (basado en HMAC)

$$\text{HOTP}(K, C) = \text{Truncate}(\text{HMAC-SHA1}(K, C))$$

Truncate extrae 32 bits de los 160 del MAC:

$$\begin{aligned} \text{offset} &= \text{mac}[19] \ \& \ 0xf && \leftarrow 4 \text{ bits del último byte} \\ \text{bin} &= \text{mac}[\text{offset} \dots \text{offset}+3] \ \& \ 0x7fffffff \\ \text{Result} &= \text{bin} \% 10^n && (n = \text{cantidad de dígitos}) \end{aligned}$$

- El **contador se incrementa en cada uso** $\Rightarrow$ cambia el código.
- Requiere **sincronización** (*look-ahead*): el servidor calcula varios intentos hacia adelante. Si se desincroniza (p. ej. pidiendo muchos códigos), suele haber que *borrar y reasociar*.
- Se aconseja **throttling** del lado del servidor (no actualiza solo).
- Genera códigos de 1 a 9 dígitos; **se recomienda al menos 6**.

10.2 TOTP — RFC 6238 (basado en tiempo)

Es lo más usado hoy (*authenticators* que cambian solos). Usa un *timestamp* en lugar del contador:

$$\text{TOTP}(K) = \text{HOTP}(K, T), \quad T = \left\lfloor \frac{\text{current unix time} - T_0}{X} \right\rfloor$$

- T_0 : tiempo de referencia (*usualmente* $T_0 = 0$, el *epoch* Unix: 1/1/1970 00:00:00).
- X : segundos de vida de cada código (*usualmente* $X = 30$).
- **No requiere contador**. Se aconseja un **drift-window** (cantidad de *ticks* desfasados aceptados, hacia atrás y adelante) por des-sincronización de relojes, más **throttling**.
- Genera códigos de 1 a 9 dígitos; **se recomienda al menos 6** (la complejidad de fuerza bruta es la misma que en HOTP: depende solo de la cantidad de dígitos).

11 Autenticación multifactor (MFA)

Consiste en requerir **dos o más factores de distinto tipo**. *Racional*: cada tipo de factor requiere comprometer **distintos assets**, lo que dificulta al atacante comprometerlos a la vez. Lo ideal es combinar factores de **distinta naturaleza**:

Combinación	Ejemplo
Algo que conozco + algo que tengo	contraseña + SMS/WhatsApp/OTP al celular
Algo que conozco + algo que soy	contraseña + Face ID / huella
Algo que tengo + algo que soy	el celular (concentra ambos)

El celular concentra dos factores... y el riesgo

El celular combina “algo que tengo” (el dispositivo) y “algo que soy” (la cara/huella leída por él). Es *ventajoso* (dos factores en un dispositivo) pero *endeblo*: si **comprometen/clonan el celular**, caen **ambos factores a la vez**.

Step-up authentication. En la práctica se suele pedir **un solo factor** (p. ej. la clave) y **eleva**r a un segundo factor solo ante **anomalías de contexto** (geolocalización distinta, primer ingreso tras mucho tiempo). Se enrojan múltiples factores pero se piden selectivamente, balanceando **seguridad y usabilidad**. La exigencia varía según la **criticidad de los activos** detrás del usuario.

12 Autenticación remota y federada (SSO)

Muchas empresas **delegan** la autenticación en sistemas externos especializados (más robustos) para enfocarse en su negocio. Implica una **relación de confianza**.

- **Productos propietarios:** Active Directory Federation Services (ADFS), CAS, Siteminder. *Active Directory* es el estándar de Microsoft (p. ej. los usuarios del ITBA viven en AD como recursos).
- **Tecnologías abiertas:**
 - **OAuth 2:** protocolo de *autorización*.
 - **OpenID Connect (OIDC):** agrega la **capa de autenticación** sobre OAuth 2 (saber *quién* sos: nombre, foto). OpenID 1 y 2 quedaron obsoletos. Es lo que hay detrás de “Iniciar sesión con Google/Facebook/Apple”.
 - **SAML** (*Security Assertion Markup Language*): estándar para intercambiar datos entre IdP y SP (autenticar / comunicar que está autenticado).

SSO y Autenticación Federada

SSO (*Single Sign On*): un **único inicio de sesión** habilita el acceso a **múltiples aplicaciones**. En la **autenticación federada** colaboran:

- **Proveedores de Identidad (IdPs):** autentican al usuario.
- **Proveedores de Servicios (SPs):** confían en el IdP.

El SP **no** necesita crear un usuario propio: se integra con el sistema de autenticación del IdP (login federado).

Autenticación federada en el S.O.

Un único conjunto de credenciales permite acceder a múltiples *servidores*.

- **LDAP** (*Lightweight Directory Access Protocol*): permite búsquedas y **almacena información de autenticación**.
- **Active Directory** usa LDAP.

Lectura recomendada

Capítulos 12–13

Computer Security: Art and Science

Matt Bishop

Cubre estos temas y profundiza más (cap. 12 — *Authentication*; cap. 13 — *Design Principles* relacionados). Para SSO/federación, los temas de OAuth2/OIDC/SAML exceden el libro y corresponden al estado del arte actual.

*Apunte integral de la Clase 7 — Autenticación · Criptografía y Seguridad, ITBA.
Síntesis de teoría (transcripciones + diapositivas) y práctica.*